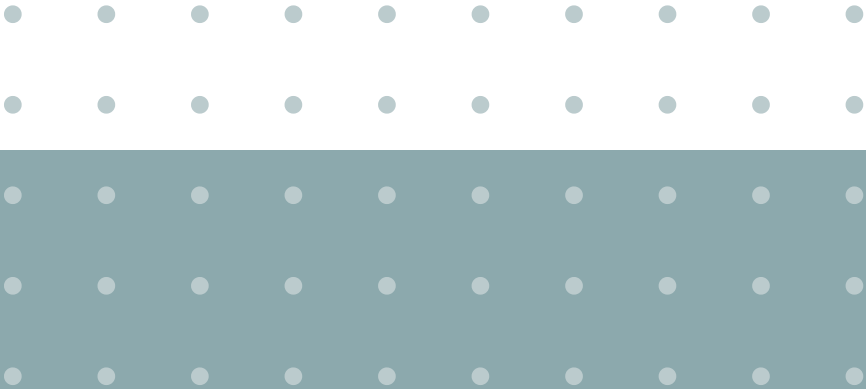




Apache Spark

Dr. Sirasit Lochanachit

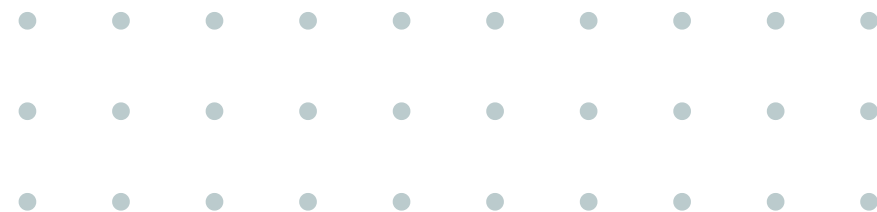
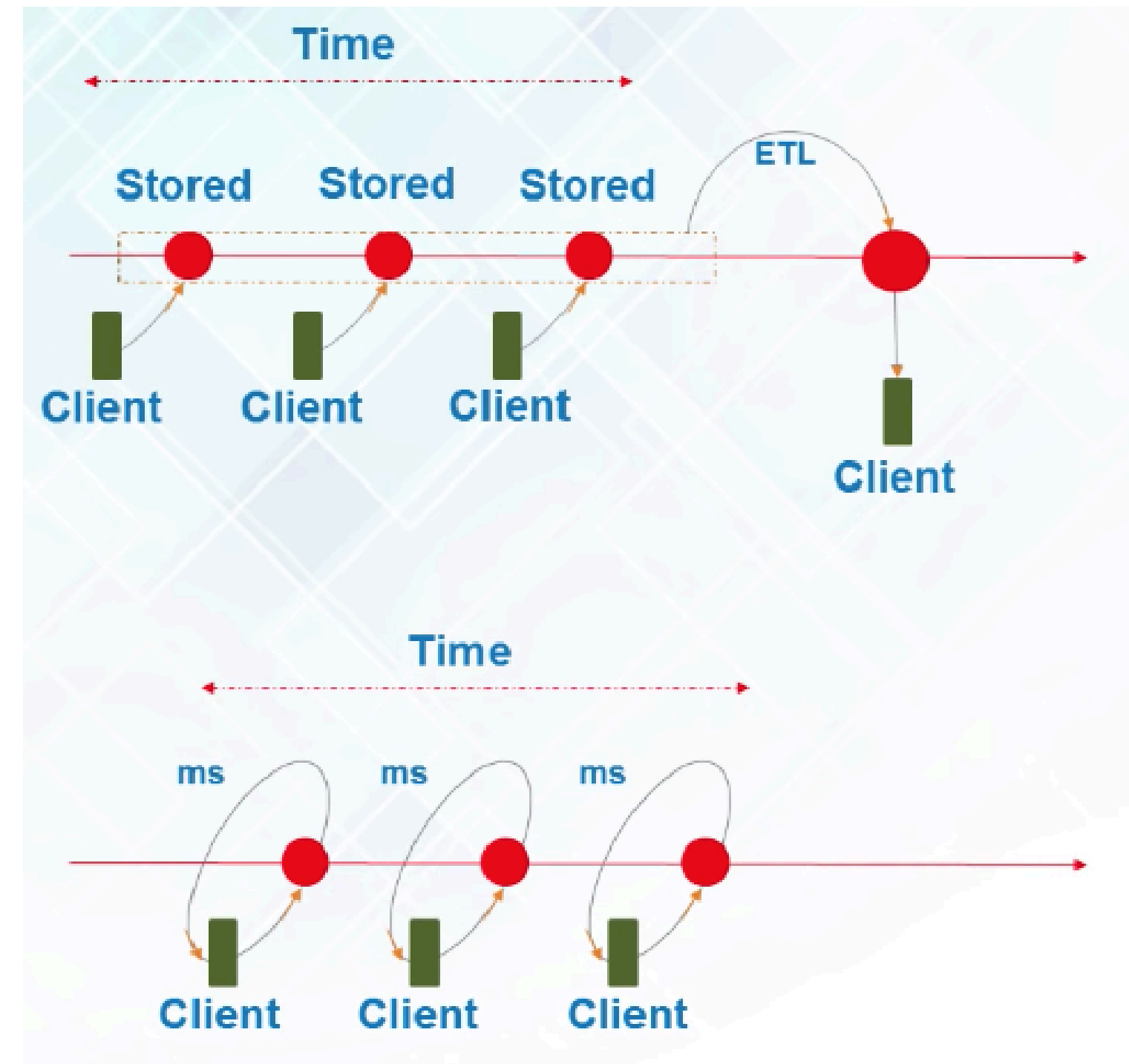
06026213 BIG DATA SYSTEMS



“What are the limitation of MapReduce?”



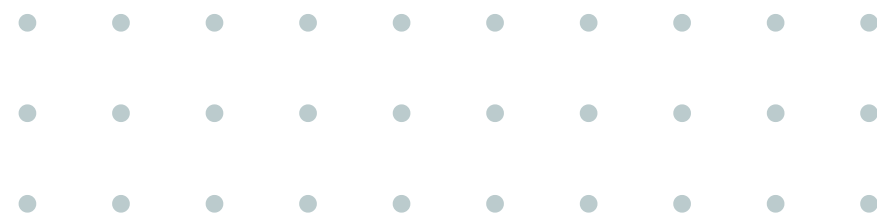
BATCH VS REAL-TIME



01. Apache Spark

02. Resilient Distributed Datasets
(RDD)

TABLE OF CONTENT

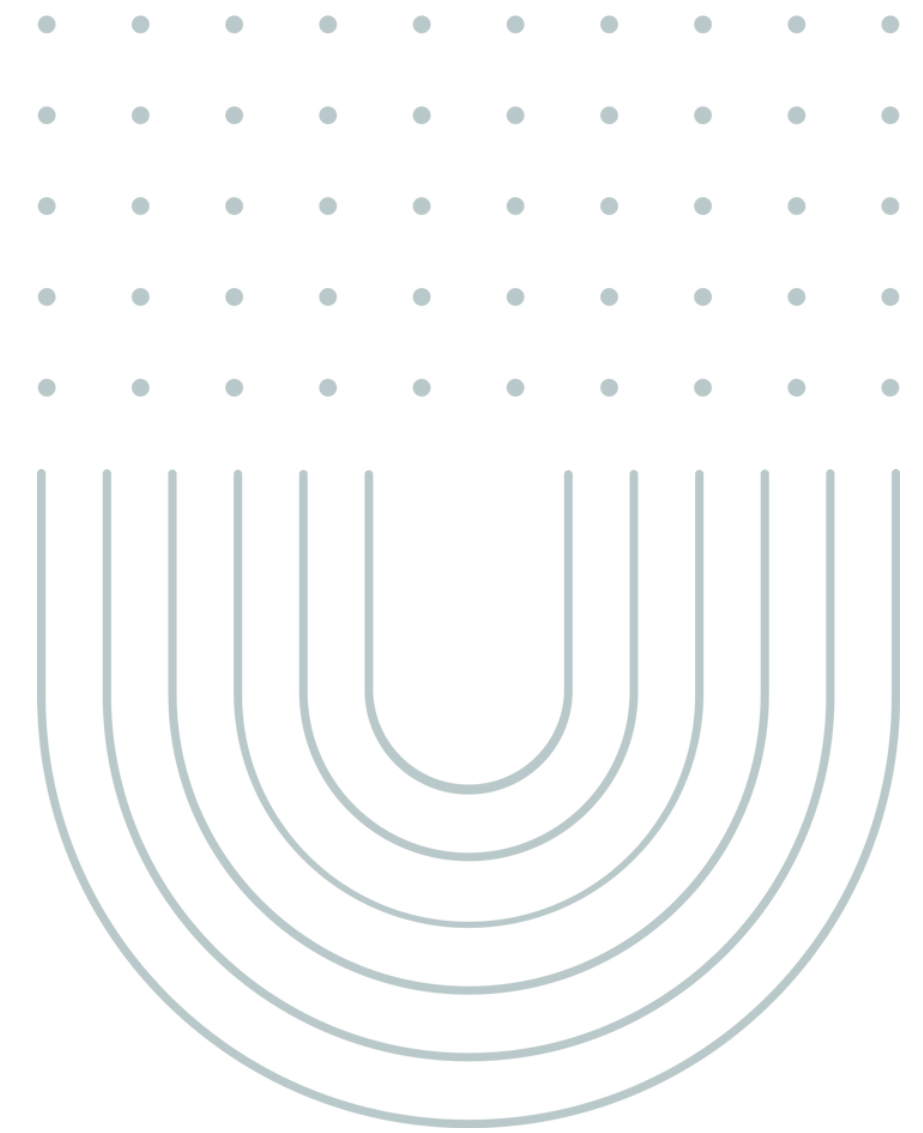


01.

APACHE SPARK

06026213 BIG DATA SYSTEMS

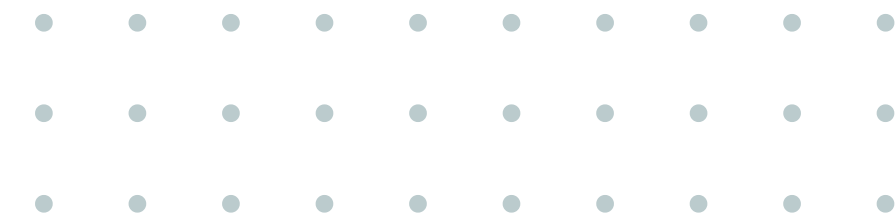
“Apache Spark is an open-source, **distributed computing system** that provides an optimized and **unified analytics engine** for large-scale data processing.”



**APACHE
SPARK**

Origin of Apache Spark

- Developed at UC Berkeley:
 - Spark was created in 2009 at the UC Berkeley AMPLab to address the inefficiencies of MapReduce.
- Goals:
 - Provide a **unified engine for big data processing**.
 - Overcome the limitations of MapReduce by **optimizing in-memory computation**.
- Open-Sourcing and Adoption:
 - Open-sourced in 2010
 - Became an Apache Software Foundation project in 2013.

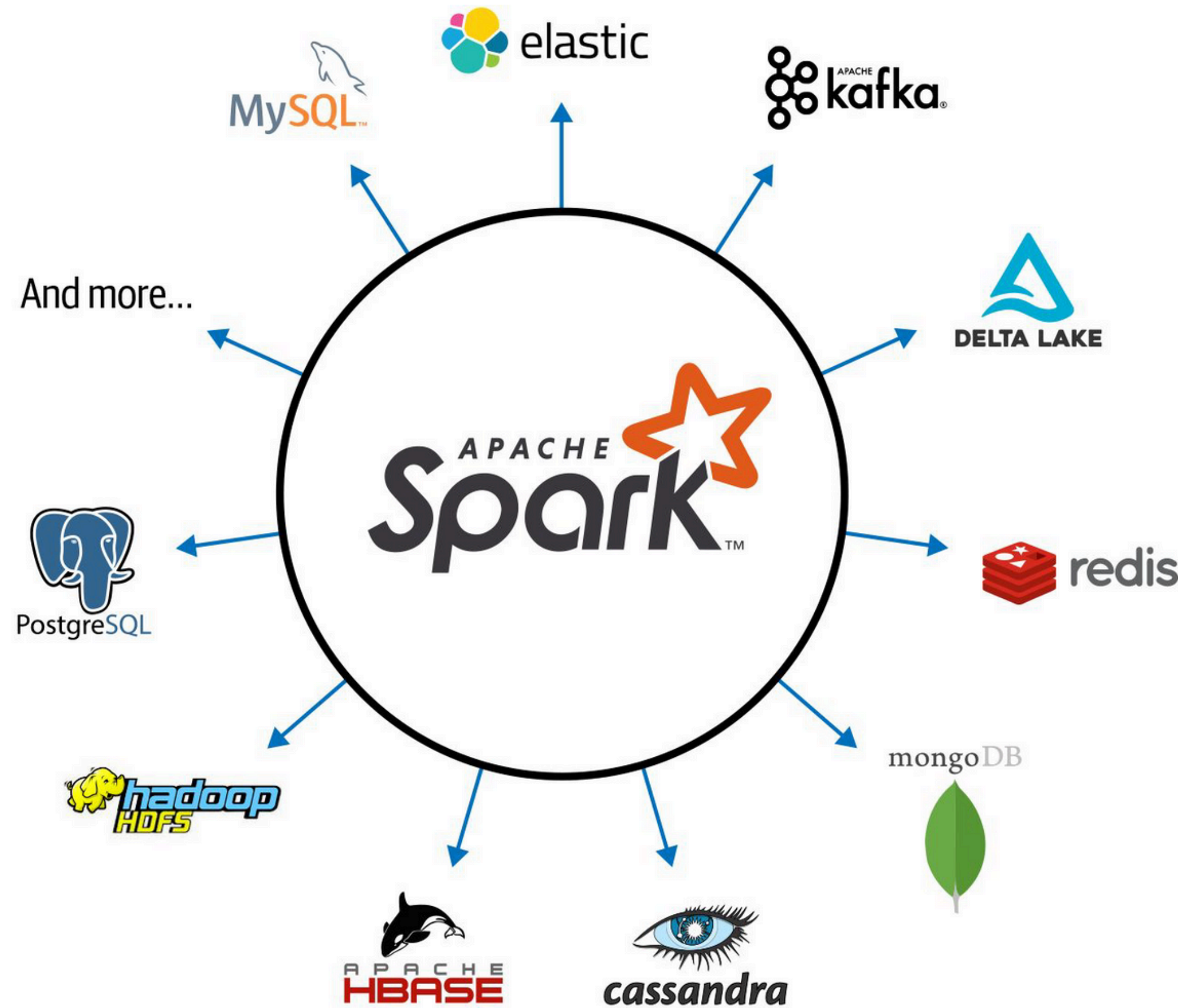




Key Features of Apache Spark

- Unified Data Processing
- In-Memory Computing
- Ease of Use
- Extensive Libraries
- Scalability

SPARK DATA SOURCES





SPARK
ECOSYSTEM

KEY DIFFERENCES

MapReduce

Apache Spark

Processing Type

Latency

Iterative Algorithm
Support

Programming
Complexity

Ecosystem
Integration

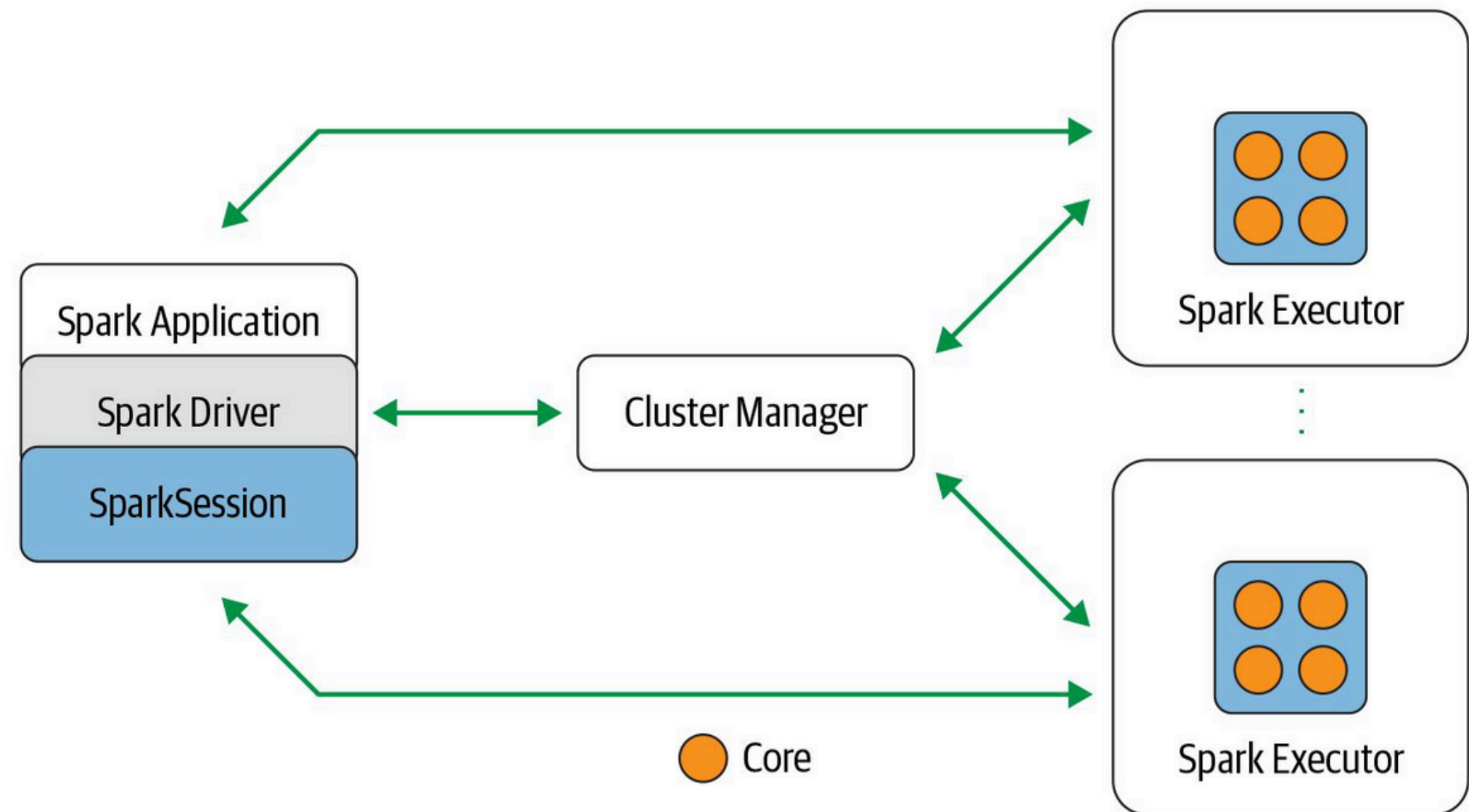
“Can Hadoop be replaced with Apache Spark
as a storage system?”



SPARK CLUSTER COMPONENTS

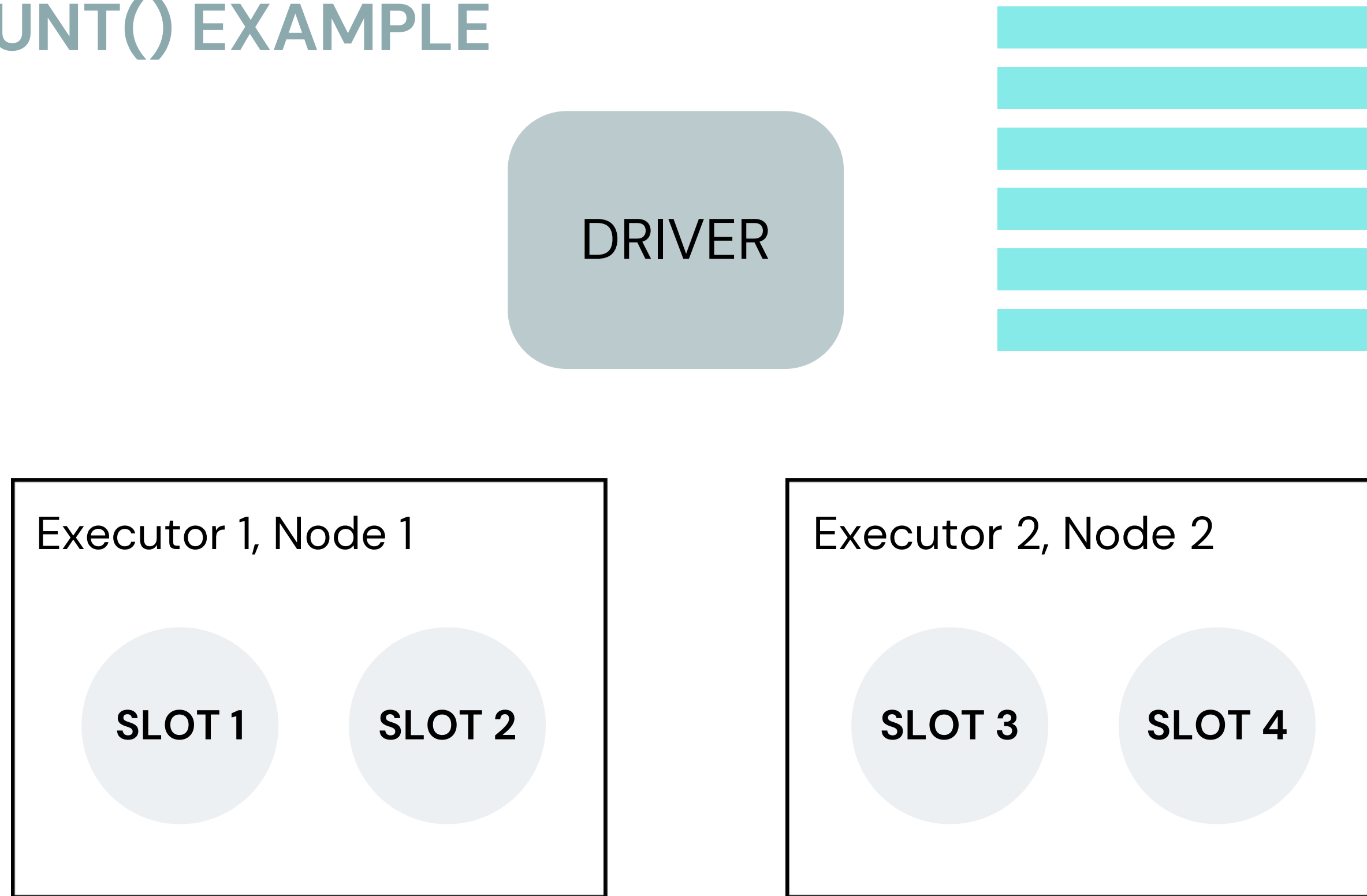
3 Core Components

- Driver
- Executor/Worker
- Cluster Manager



Source: <https://spark.apache.org/docs/3.5.3/cluster-overview.html>

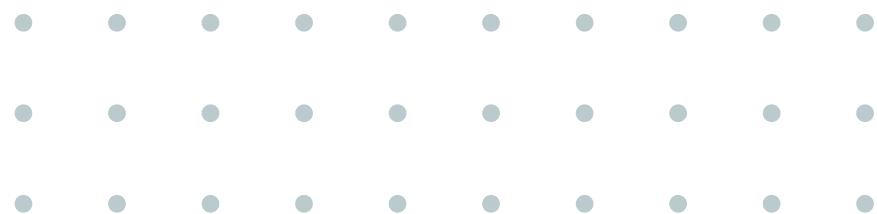
COUNT() EXAMPLE





Key points to remember

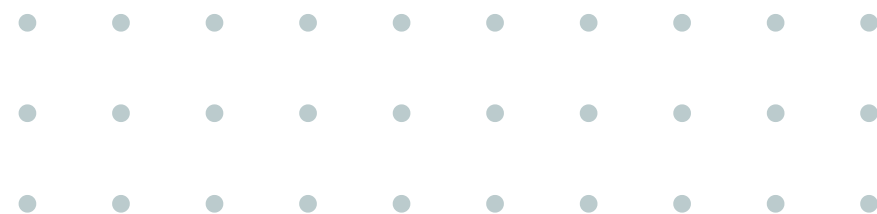
- Driver plan job(s) and make sure that they are executed.
 - If an executor fails, it assign a task to another one
 - Driver, almost never, touch data.
- Executor, execute tasks and report the status
 - There can be more than 1 executor on a worker node.
- There is as many slots on executor as CPU cores





Key points to remember

- There can be as many tasks executed by executor in parallel as many slots
- Data are being splitted into partitions
 - 1 Partition for 1 task
- Driver and executors process run on the Java Virtual Machine (JVM)

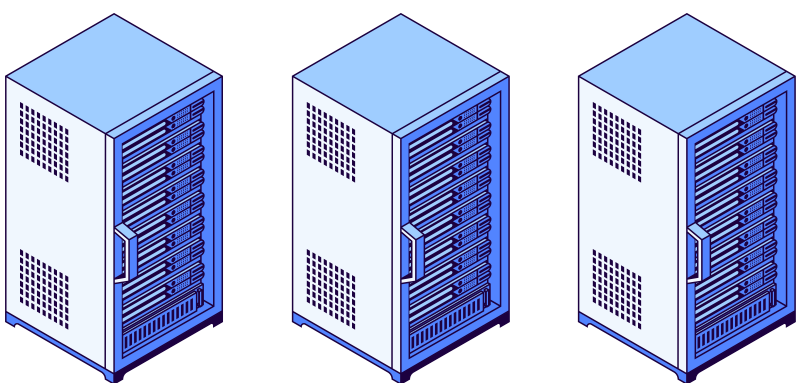


Deployment Modes

Your Computer



Data Center / Cloud



Local

Driver and
executors

Client

Driver

Executors

Cluster

Driver and
executors

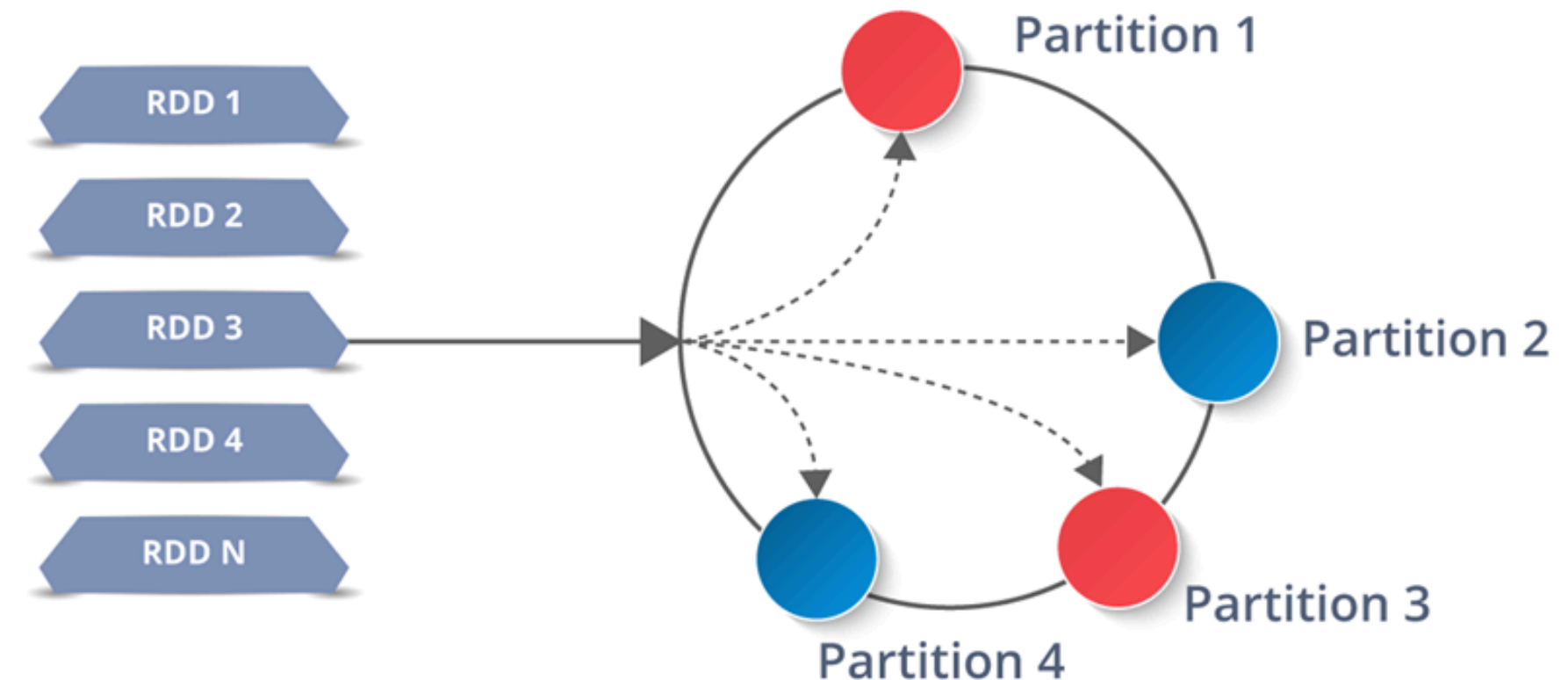
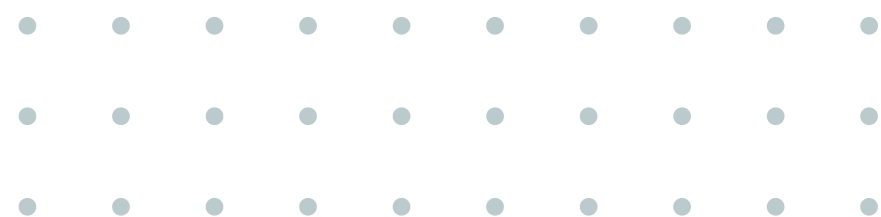
02.

RESILIENT DISTRIBUTED DATASETS (RDD)

06026213 BIG DATA SYSTEMS

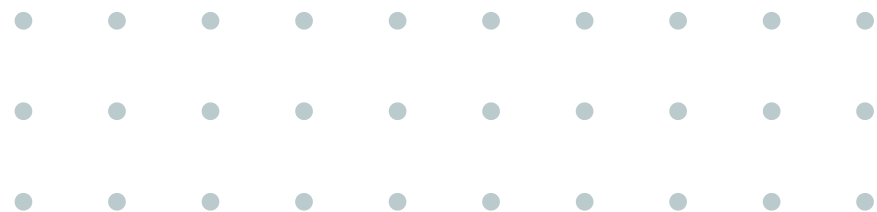
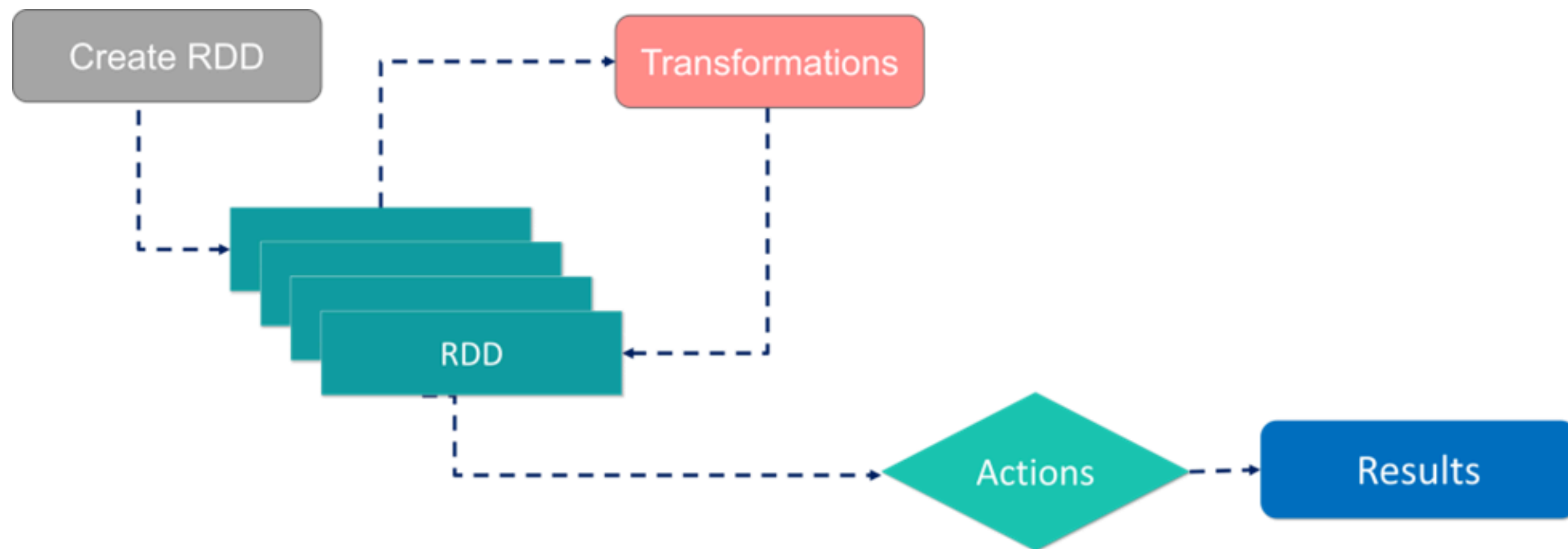
WHAT IS RDD?

- A **fault-tolerant** collection of elements that can be operated on **in parallel**
- Resilient: Fault tolerant and is capable of rebuilding data on failure
- Distributed: Distributed data among the multiple nodes in a cluster
- Dataset: Collection of partitioned data with values



CHARACTERISTICS OF RDD

- Low-level API
- Immutable
- Follows **lazy transformation**



Transformations

- Transformations create a new dataset from an existing one
- All transformations in Spark are **lazy**
 - They do not compute their results right away – instead, they remember the transformations applied to some base dataset
- Optimize the required calculations
- Support recovering from lost data partitions



Transformations

transformation	description
map (<i>func</i>)	return a new distributed dataset formed by passing each element of the source through a function <i>func</i>
filter (<i>func</i>)	return a new dataset formed by selecting those elements of the source on which <i>func</i> returns true
flatMap (<i>func</i>)	<u>similar to map</u> , but each input item can be mapped to 0 or more output items (so <i>func</i> should return a Seq rather than a single item)
sample (<i>withReplacement</i> , <i>fraction</i> , <i>seed</i>)	sample a fraction <i>fraction</i> of the data, with or without replacement, using a given random number generator <i>seed</i>
union (<i>otherDataset</i>)	return a new dataset that contains the union of the elements in the source dataset and the argument
distinct (<i>[numTasks]</i>)	return a new dataset that contains the distinct elements of the source dataset

Transformations

transformation	description
groupByKey ([<i>numTasks</i>])	when called on a dataset of (K, V) pairs, returns a dataset of (K, Seq[V]) pairs
reduceByKey (<i>func</i> , [<i>numTasks</i>])	when called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function
sortByKey ([<i>ascending</i>] , [<i>numTasks</i>])	when called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean ascending argument
join (<i>otherDataset</i> , [<i>numTasks</i>])	when called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key
cogroup (<i>otherDataset</i> , [<i>numTasks</i>])	when called on datasets of type (K, V) and (K, W), returns a dataset of (K, Seq[V], Seq[W]) tuples – also called groupWith
cartesian (<i>otherDataset</i>)	when called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements)

Actions

- take()
- collect()
- count()
- reduce()



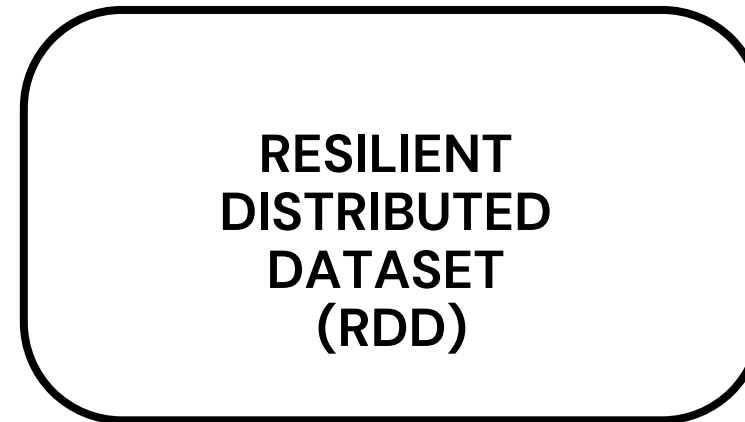
Caching & Persistence

- Caching: Store RDD in-memory
- Persistence: either in-memory or on disk

Persistence

<i>transformation</i>	<i>description</i>
MEMORY_ONLY	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed. This is the default level.
MEMORY_AND_DISK	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
MEMORY_ONLY_SER	Store RDD as serialized Java objects (one byte array per partition). This is generally more space-efficient than deserialized objects, especially when using a fast serializer, but more CPU-intensive to read.
MEMORY_AND_DISK_SER	<u>Similar to</u> MEMORY_ONLY_SER, but spill partitions that don't fit in memory to disk instead of recomputing them on the fly each time they're needed.
DISK_ONLY	Store the RDD partitions only on disk.
MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc	Same as the levels <u>above</u> , but replicate each partition on two cluster nodes.

Data



DRIVER

Executor 1, Node 1

SLOT 1

SLOT 2

Partition 1

Partition 2

Executor 2, Node 2

SLOT 3

SLOT 4

Partition 3

Partition 4